Build with Gemini 4 Argon: 1-day hackathon builder guide

You have one day to build a working application or agent with Gemini 4 Argon (`gemini-4-argon`), our new frontier model for software engineering, complex knowledge work, cybersecurity defense and multimodal creation.Gemini 4 Argon unifies input, thinking and output in a single 1M-token context window. It works longer on its own, retains context across large codebases and dense document sets.

What makes a strong 1-day project

Judges score working demos, not slide decks. Strong projects pass 4 tests:

- it solves a specific problem for a real person you can name, rather than a generic audience
- it does work that needed Gemini 4 Argon: long-horizon coding, autonomous verification loops, multi-file synthesis, 3D and WebGPU generation, or multi-agent orchestration - it keeps people in control when taking actions that affect external systems, money or private data
- it runs live during judging with real inputs in under 3 minutes

Your toolkit: Gemini 4 Argon and companion models

Pick each model by the job it does in your application. Most teams pair Gemini 4 Argon as the lead brain or orchestrator with one or two fast companion models for voice, images, audio or parallel subtasks.

Job in your app	Model or agent	Model or agent ID	When to use it
Frontier reasoning, long-horizon coding, 3D builds and orchestration	Gemini 4 Argon	`gemini-4-argon`	Complex architecture, multi-file code generation, security patching, agentic video analysis, Three.js/WebGPU scenes and lead orchestration across a shared 1M-token input and output window.
Autonomous sandbox execution	Antigravity managed agent	`antigravity-preview-09-2 026`	One API call starts a Google-hosted Linux sandbox where Argon writes code, runs tests, manages files, calls MCP tools and searches the web.

Job in your app	Model or agent	Model or agent ID	When to use it
Fast parallel subagents and tool calls	Gemini 3.8 Flash	`gemini-3.8-flash`	High-speed workhorse for parallel leaf tasks, fast codebase exploration, boilerplate tests and low-latency tool calls orchestrated by Argon.
High-frequency routing and checks	Gemini 3.5 Flash-Lite	`gemini-3.5-flash-lite`	Lowest-latency classification, intent routing and guardrails.
Deep web research with citations	Deep Research agent	`deep-research-preview-04 -2026`	Multi-step background research that returns a cited report.
Real-time voice and camera conversation	Gemini 3.8 Live	`gemini-3.8-live`	Natural two-way voice and vision sessions with interruptions and non-blocking function calls. Use `gemini-3.8-live-extended -thinking` when the voice agent must reason deeply while talking.
Live speech-to-text and translation	Gemini 3.5 Transcribe and Live Translate	`gemini-3.5-transcribe-live ve`, `gemini-3.5-live-translat e-preview`	Streaming captions (`gemini-3.5-transcribe -l ive`), recorded audio with speaker labels (`gemini-3.5-transcribe`), or continuous speech-to-speech translation across more than 70 languages.
Expressive speech, voice design and replication	Gemini 3.8 Flash TTS	`gemini-3.8-flash-tts`	Studio-quality speech in 130 languages with designed or replicated voices. Use `gemini-3.8-flash-lite-tt s` for faster output.

UI assets, textures, sprites and 4K images	Nano Banana 2.1	`gemini-nano-banana-2.1`	High-consistency image generation, multi-turn editing, accurate text rendering, Search grounding and panoramic ratios up to 8:1. Use `gemini-3.1-flash-lite-image` (Nano Banana 2 Lite) for sub-second speed.

Job in your app	Model or agent	Model or agent ID	When to use it
Video generation with native audio	Gemini Omni Flash	`gemini-omni-1.1-flash`	Generates and conversationally edits video with synchronized sound from text, images, audio or video. Run in the background.
Songs, vocals and background loops	Lyria 3.5	`lyria-3.5`	Full songs with vocals and timed lyrics in 44.1 kHz stereo, or 30-second loops with `lyria-3-clip-preview`.
Private or on-device open models	Gemma 4	`gemma-4-31b-it`	Open weights available on the Interactions API, Kaggle and Hugging Face. The E2B and E4B sizes run locally on phones and laptops with native audio input.

Get started in 15 minutes

1. Claim your pre-provisioned account from the Google team at kickoff
2. Install the official Gemini API skills in your coding agent so it writes current `gemini-4-argon` and Interactions API code, or prototype directly in Google AI Studio Build and Google Antigravity.
3. Run your first `gemini-4-argon` call on the Interactions API using `google-genai` or `@google/genai` version 2.3.0 or later.
4. Reach out to the Google mentors onsite whenever you hit a blocker.

Give your coding agent the Gemini API skills

Coding agents trained on older data often suggest deprecated SDKs or removed parameters. Install the official skills so your agent knows `gemini-4-argon`, the Interactions API and Managed Agents:

```shell
npx skills add google-gemini/gemini-skills --skill gemini-api-dev npx skills add google-gemini/gemini-skills --skill gemini-live-api-dev agy plugin install https://github.com/google-gemini/gemini-skills
```

What to know when calling Gemini 4 Argon

The Interactions API is the recommended endpoint for new projects. One API covers both models and managed agents, supports server-side state (`previous_interaction_id`) and handles background execution.

1. Control reasoning depth with `thinking_level`

Gemini 4 Argon uses `thinking_level` (`"low"` | `"medium"` | `"high"`) to control how much reasoning effort the model spends before responding:

- `"medium"` (default): balances speed, cost and first-pass accuracy for interactive coding and agentic tool use
- `"high"`: maximizes reasoning depth for multi-file debugging, security vulnerability patching, formal maths and upfront system design
- `"low"`: minimizes latency and cost for fast classification, routing and targeted extraction

```python
from google import genai

client = genai.Client()

interaction = client.interactions.create(
 model="gemini-4-argon",
 input="Build a single-file HTML drum machine and synth using the Web Audio API with a 16-step sequencer, tempo slider, and a live waveform visualizer.",
generation_config={
 "thinking_level": "medium" # "low" | "medium" (default) | "high"  },
)
print(interaction.output_text)
```

2. Shared 1M-token context window and long decode (`background=True`)

Earlier models capped total output (thinking plus response) at 64K tokens. In Gemini 4 Argon, input, thinking and output share a single 1M-token window:

- default output limit is 65,536 (64K) tokens on the Interactions API for both streaming and non-streaming calls with no code changes needed
- to generate past 65,536 tokens (up to 1M tokens minus your input), set `background=True` and set `max_output_tokens` above `65536` on the Interactions API

Set `max_output_tokens` high enough that deep thinking does not cut off your multi-file diff or proof mid-output, and low enough to cap spend on open-ended prompts:

Python

```
import time
from google import genai

client = genai.Client()

interaction = client.interactions.create(
 model="gemini-4-argon",
 input="Write a complete multi-module physics simulation and test suite.",
background=True,
 generation_config={
 "max_output_tokens": 262144,
 "thinking_level": "high",
 },
)

while interaction.status in ("in_progress", "queued"):
 time.sleep(5)
 interaction = client.interactions.get(id=interaction.id)

print(interaction.output_text)
```

3. Clean up legacy parameters

If you are adapting code written for Gemini 2.5 or Gemini 3 models, remove these deprecated settings when calling `gemini-4-argon`:

- replace `thinking_budget` with `thinking_level` (`thinking_budget` is deprecated and returns a warning)
- remove `temperature`, `top_p` and `top_k` (custom sampling values are ignored by `gemini-4-argon` and return a deprecation warning)
- remove explicit `cached_content` calls (implicit context caching runs automatically at a 95% discount on cached input tokens; introductory pricing through 30 November 2026 is $2 per 1M input tokens and $10 per 1M output tokens)
- increase HTTP client timeouts to 15 to 20 minutes for complex `"high"` thinking prompts, or run them asynchronously with `background=True`

4. Run Gemini 4 Argon inside Managed Agents

You can pass `"model": "gemini-4-argon"` to the Antigravity managed agent (`antigravity-preview-09-2026`). Google provisions a hosted sandbox where Argon plans, writes code, verifies its own output, searches the web and calls MCP tools in a single API call:

```python
from google import genai

client = genai.Client()

interaction = client.interactions.create(
 agent="antigravity-preview-09-2026",
 agent_config={
 "type": "antigravity",
 "model": "gemini-4-argon",
 },
 input=(
 "Audit https://web.dev for performance, Core Web Vitals, and SEO. "
"Query Google's PageSpeed Insights API for both Mobile and Desktop
strategies. "
 "Check search indexing with Google Search for site:web.dev. "  "Format the
output as a side-by-side scorecard table with prioritized fixes."
 ),
 environment="remote",
)
print(interaction.output_text)
```

When your agent needs API tokens (such as GitHub or Stripe), register them with the Credentials API (`client.credentials.create`) and restrict the sandbox network allowlist (`environment={"type": "remote", "network": {"allowlist": [...]}}`) so secrets are injected at the proxy and never enter the model context.

5 patterns that save hours in a 1-day hackathon

1. Align on intent before long runs. Share your goal with Argon and ask it to surface clarifying questions, edge cases and architecture trade-offs before writing code. Once you agree on the plan and confirm it can reach your tools and docs, step back and let Argon execute the multi-file build to completion instead of prompting turn by turn.
2. Give Argon a verification loop. Argon performs best when it can test its own changes against a concrete pass or fail signal. Point it at your test suite, compiler output or runtime logs, or ask it to write a headless Chrome verification script first, record a baseline score, and iterate in a loop until every check passes with zero regressions. 3. Use multimodal references and agentic video understanding. Pass 1 or 2 reference images when building frontends or 3D scenes (Blender, Three.js, WebGPU), or ask

Argon to extract typography, color palette and layout rules into a `DESIGN.md` file first. When debugging UI or timing bugs, record a short screen-share talking through the issue alongside your console logs; Argon's agentic video understanding scrubs across timestamps, inspects high-resolution frames and patches the underlying components.

4. Orchestrate Argon with Flash and media subagents. Use `gemini-4-argon` (`thinking_level="medium"` or `"high"`) as the lead architect to decompose the problem, define interfaces and review diffs, while delegating parallel leaf modules and high-frequency checks to `gemini-3.8-flash` subagents. Let Argon call `gemini-nano-banana-2.1` for UI textures and sprites, `gemini-3.8-live` for real-time voice interaction, and `gemini-3.8-flash-tts` or `lyria-3.5` for voice and music.
5. Capture codebase quirks and visual explainers. When Argon solves a non-obvious setup or build quirk, ask it to write the lesson into `AGENTS.md` or a reusable skill so your team never hits the same snag twice. Before your final demo, ask Argon to generate an interactive HTML architecture diagram or pipeline walkthrough of what you built.

Resources and feedback

- Gemini 4 Argon developer guide: aistudio.google.com/learn/gemini-4-argon-developer-guide
- Google AI Studio Playground and Build mode: aistudio.google.com and aistudio.google.com/build
- Google Antigravity: antigravity.google
- Gemini API skills for coding agents: github.com/google-gemini/gemini-skills - Gemini API documentation and cookbook: ai.google.dev/gemini-api/docs and github.com/google-gemini/cookbook
- Live API starter apps: github.com/google-gemini/gemini-live-api-examples

The Google team is onsite all day.

Tell us what worked, what broke and what you wish existed, or email gemini-developer-feedback@google.com.